Moscow Aviation Institute
(National Research University)

Faculty of Information Technologies and Applied Mathematics

Department of Computational Mathematics and Programming

Laboratory work No. 1, course "Numerical Methods"

Numerical Methods of Linear Algebra

Student: M. A. Khorokhorin
Teacher:
Group: M8O-306BV-24
Variant: 28
Date: 30.09.2026
Grade:
Signature:

Moscow, 2026

1 LU decomposition

1.1 Task

Implement the LU decomposition of a matrix with the choice of the pivot element. Use it to solve the system of linear algebraic equations, to compute the determinant and the inverse matrix.

$$\begin{cases} x_1 + 4x_2 - 9x_3 + 7x_4 = -67 \\ 2x_1 - 2x_2 - 2x_3 + 3x_4 = -57 \\ -x_1 + 3x_2 - 9x_3 - x_4 = -26 \\ -5x_1 + 2x_2 + 2x_3 + x_4 = 52 \end{cases}$$

1.2 Method

The matrix is factored as $PA = LU$, where L is a unit lower triangular matrix, U is an upper triangular matrix and P is a permutation matrix. At step $k = 1, \dots, n$ the row with the largest $|u_{ik}|$, $i \geq k$, is swapped with row k (partial pivoting), and then

$$l_{ik} = \frac{u_{ik}}{u_{kk}}, \qquad u_{ij} \leftarrow u_{ij} - l_{ik}u_{kj}, \qquad i, j > k.$$

Partial pivoting keeps $|l_{ik}| \leq 1$ and allows zeros on the diagonal of A. A pivot $|u_{kk}| \leq 10^{-12}\|A\|_\infty$ means that the matrix is singular to working precision: a relative perturbation of A comparable to rounding errors can make it exactly singular.

The system $Ax = b$ is solved by two triangular substitutions, $Lz = Pb$ and $Ux = z$:

$$z_i = (Pb)_i - \sum_{j<i} l_{ij} z_j, \qquad x_i = \frac{1}{u_{ii}}\Big(z_i - \sum_{j>i} u_{ij} x_j\Big).$$

The determinant is $\det A = (-1)^s \prod_{i=1}^{n} u_{ii}$, where s is the number of row swaps. The j-th column of A^{-1} is the solution of $Ax = e_j$; all n systems reuse the same factorization, so the decomposition costs $O(n^3)$ once and every solution costs $O(n^2)$.

1.3 Results

The program reads n, the matrix A and the vector b and prints P as the order of rows of A, the factors L and U, the solution, the determinant and the inverse matrix.

Listing 1: task1/input.txt

```
4
1 4 -9 7
2 -2 -2 3
-1 3 -9 -1
-5 2 2 1
-67 -57 -26 52
```

Listing 2: task1/output.txt

```
P = (4, 1, 3, 2)
L =
   1.0000   0.0000   0.0000   0.0000
  -0.2000   1.0000   0.0000   0.0000
   0.2000   0.5909   1.0000   0.0000
  -0.4000  -0.2727   0.8211   1.0000
U =
  -5.0000   2.0000   2.0000   1.0000
   0.0000   4.4000  -8.6000   7.2000
   0.0000   0.0000  -4.3182  -5.4545
   0.0000   0.0000   0.0000   9.8421
x = (-5.0000, 9.0000, 7.0000, -5.0000)
det(A) = 935.0000
A^-1 =
   0.1027  -0.1979  -0.1112  -0.2364
   0.1786  -0.4171  -0.1102  -0.1091
   0.0396  -0.1283  -0.1262  -0.0182
   0.0770   0.1016  -0.0834   0.0727
```

The solution is $x = (-5, 9, 7, -5)^T$ and $\det A = 935$. The unit tests check $LU = PA$, $|l_{ik}| \le 1$ and $AA^{-1} = E$ on random matrices up to 8×8.

1.4 Source code

Listing 3: task1/lu.py

```python
from dataclasses import dataclass

from lab1.common import (
    Matrix,
    Vector,
    copy_matrix,
    identity,
```

```
    is_square,
    matrix_norm_inf,
    zeros,
)

SINGULARITY_TOLERANCE = 1e-12


@dataclass(frozen=True, slots=True)
class LuDecomposition:
    lower: Matrix
    upper: Matrix
    permutation: list[int]
    sign: int


def decompose_lu(matrix: Matrix) -> LuDecomposition:
    if not matrix or not is_square(matrix):
        raise ValueError("LU decomposition needs a non-empty square matrix")
    n = len(matrix)
    threshold = SINGULARITY_TOLERANCE * matrix_norm_inf(matrix)
    lower = identity(n)
    upper = copy_matrix(matrix)
    permutation = list(range(n))
    sign = 1

    for k in range(n):
        pivot = k
        for i in range(k + 1, n):
            if abs(upper[i][k]) > abs(upper[pivot][k]):
                pivot = i
        if abs(upper[pivot][k]) <= threshold:
            raise ValueError("matrix is singular")
        if pivot != k:
            upper[k], upper[pivot] = upper[pivot], upper[k]
            for j in range(k):
                lower[k][j], lower[pivot][j] = lower[pivot][j], lower[k][j]
            permutation[k], permutation[pivot] = permutation[pivot], permutation[k]
            sign = -sign
        for i in range(k + 1, n):
            factor = upper[i][k] / upper[k][k]
            lower[i][k] = factor
            upper[i][k] = 0.0
            for j in range(k + 1, n):
                upper[i][j] -= factor * upper[k][j]
    return LuDecomposition(lower, upper, permutation, sign)


def solve_lu(lu: LuDecomposition, rhs: Vector) -> Vector:
    n = len(lu.upper)
    if len(rhs) != n:
        raise ValueError("right-hand side size does not match the matrix")
    solution = [0.0] * n
    for i in range(n):
        total = rhs[lu.permutation[i]]
        for j in range(i):
            total -= lu.lower[i][j] * solution[j]
        solution[i] = total
    for i in reversed(range(n)):
        total = solution[i]
        for j in range(i + 1, n):
            total -= lu.upper[i][j] * solution[j]
```

```
        solution[i] = total / lu.upper[i][i]
    return solution


def determinant(lu: LuDecomposition) -> float:
    result = float(lu.sign)
    for i in range(len(lu.upper)):
        result *= lu.upper[i][i]
    return result


def inverse(lu: LuDecomposition) -> Matrix:
    n = len(lu.upper)
    result = zeros(n, n)
    for j in range(n):
        unit = [0.0] * n
        unit[j] = 1.0
        for i, value in enumerate(solve_lu(lu, unit)):
            result[i][j] = value
    return result
```

Listing 4: task1/main.py

```
import sys
from pathlib import Path

from lab1.common import InputReader, format_matrix, format_number, format_vector, run
from lab1.task1.lu import decompose_lu, determinant, inverse, solve_lu

INPUT = Path(__file__).with_name("input.txt")


def task(reader: InputReader) -> list[str]:
    n = reader.size()
    matrix = reader.matrix(n)
    rhs = reader.vector(n)
    reader.finish()

    lu = decompose_lu(matrix)
    solution = solve_lu(lu, rhs)
    inverted = inverse(lu)

    permutation = ", ".join(str(index + 1) for index in lu.permutation)
    return [
        f"P = ({permutation})",
        "L =",
        *format_matrix(lu.lower),
        "U =",
        *format_matrix(lu.upper),
        f"x = {format_vector(solution)}",
        f"det(A) = {format_number(determinant(lu))}",
        "A^-1 =",
        *format_matrix(inverted),
    ]


if __name__ == "__main__":
    sys.exit(run(task, INPUT, sys.argv[1:]))
```

2 Thomas algorithm

2.1 Task

Implement the Thomas algorithm (tridiagonal matrix algorithm) taking the non-zero elements of the matrix and the right-hand side as input. Use it to solve the system with a tridiagonal matrix.

$$\begin{cases} 16x_1 - 9x_2 = -27 \\ 8x_1 - 13x_2 - 5x_3 = -84 \\ -3x_2 - 21x_3 + 9x_4 = -225 \\ -9x_3 + 16x_4 - 5x_5 = -89 \\ x_4 - 9x_5 = 69 \end{cases}$$

2.2 Method

Row i of the system is $a_i x_{i-1} + b_i x_i + c_i x_{i+1} = d_i$ with $a_1 = c_n = 0$. The solution is sought as $x_i = P_i x_{i+1} + Q_i$. The forward sweep computes

$$P_i = \frac{-c_i}{b_i + a_i P_{i-1}}, \qquad Q_i = \frac{d_i - a_i Q_{i-1}}{b_i + a_i P_{i-1}}, \qquad P_0 = Q_0 = 0,$$

and the backward sweep gives $x_n = Q_n$ and $x_i = P_i x_{i+1} + Q_i$ for $i = n-1, \ldots, 1$. Both sweeps take $O(n)$ operations and $O(n)$ memory.

The sufficient condition of correctness and stability is $a_i \neq 0$, $c_i \neq 0$ and diagonal dominance $|b_i| \geq |a_i| + |c_i|$ for every i with a strict inequality for at least one i. Then $|P_i| \leq 1$, the denominators do not vanish and rounding errors do not grow in the backward sweep. Without the requirement $a_i, c_i \neq 0$ dominance is not enough: the system can split into independent blocks, one of which is singular. The program checks this condition and reports it, but it also solves systems that do not satisfy it as long as no denominator is zero.

2.3 Results

The program reads n, the sub-diagonal $a_2, \ldots, a_n$, the diagonal $b_1, \ldots, b_n$, the super-diagonal $c_1, \ldots, c_{n-1}$ and the right-hand side $d_1, \ldots, d_n$.

Listing 5: task2/input.txt

```
5
8 -3 -9 1
16 -13 -21 16 -9
-9 -5 9 -5
-27 -84 -225 -89 69
```

Listing 6: task2/output.txt

```
x = (0.0000, 3.0000, 9.0000, -3.0000, -8.0000)
Stability condition: yes
```

The solution is $x = (0, 3, 9, -3, -8)^T$. The matrix of the variant is diagonally dominant, the second row turns the condition into an equality: $|-13| = |8| + |-5|$. The unit tests compare the Thomas algorithm with the LU decomposition on random systems up to 12×12.

2.4 Source code

Listing 7: task2/tridiagonal.py

```python
from dataclasses import dataclass

from lab1.common import Vector

SINGULARITY_TOLERANCE = 1e-12


@dataclass(frozen=True, slots=True)
class TridiagonalSystem:
    lower: Vector
    diagonal: Vector
    upper: Vector
    rhs: Vector


def _validate(system: TridiagonalSystem) -> None:
    n = len(system.diagonal)
    if n == 0 or len(system.rhs) != n or len(system.lower) + 1 != n or len(system.upper) + 1 != n:
        raise ValueError("tridiagonal system has inconsistent sizes")


def _lower_at(system: TridiagonalSystem, row: int) -> float:
    return 0.0 if row == 0 else system.lower[row - 1]


def _upper_at(system: TridiagonalSystem, row: int) -> float:
    return 0.0 if row + 1 == len(system.diagonal) else system.upper[row]

```

```python

def solve_tridiagonal(system: TridiagonalSystem) -> Vector:
    _validate(system)
    n = len(system.diagonal)
    scale = 0.0
    for i in range(n):
        scale = max(
            scale,
            abs(_lower_at(system, i)),
            abs(system.diagonal[i]),
            abs(_upper_at(system, i)),
        )

    p = [0.0] * n
    q = [0.0] * n
    for i in range(n):
        a = _lower_at(system, i)
        previous_p = 0.0 if i == 0 else p[i - 1]
        previous_q = 0.0 if i == 0 else q[i - 1]
        denominator = system.diagonal[i] + a * previous_p
        if abs(denominator) <= SINGULARITY_TOLERANCE * scale:
            raise ValueError("zero denominator in the Thomas algorithm")
        p[i] = -_upper_at(system, i) / denominator
        q[i] = (system.rhs[i] - a * previous_q) / denominator

    solution = [0.0] * n
    solution[n - 1] = q[n - 1]
    for i in reversed(range(n - 1)):
        solution[i] = p[i] * solution[i + 1] + q[i]
    return solution


def meets_stability_condition(system: TridiagonalSystem) -> bool:
    _validate(system)
    if 0.0 in system.lower or 0.0 in system.upper:
        return False
    strict = False
    for i in range(len(system.diagonal)):
        diagonal = abs(system.diagonal[i])
        off_diagonal = abs(_lower_at(system, i)) + abs(_upper_at(system, i))
        if diagonal < off_diagonal:
            return False
        strict = strict or diagonal > off_diagonal
    return strict
```

Listing 8: task2/main.py

```python
import sys
from pathlib import Path

from lab1.common import InputReader, format_vector, run
from lab1.task2.tridiagonal import TridiagonalSystem, meets_stability_condition, solve_tridiagonal

INPUT = Path(__file__).with_name("input.txt")


def task(reader: InputReader) -> list[str]:
    n = reader.size()
    lower = reader.vector(n - 1)
    diagonal = reader.vector(n)
    upper = reader.vector(n - 1)
```

```
    rhs = reader.vector(n)
    reader.finish()

    system = TridiagonalSystem(lower, diagonal, upper, rhs)
    solution = solve_tridiagonal(system)

    stable = "yes" if meets_stability_condition(system) else "no"
    return [f"x = {format_vector(solution)}", f"Stability condition: {stable}"]


if __name__ == "__main__":
    sys.exit(run(task, INPUT, sys.argv[1:]))
```

3 Simple iteration and Seidel methods

3.1 Task

Implement the simple iteration method and the Seidel method taking the matrix, the right-hand side and the precision as input. Solve the system and analyze the number of iterations needed to reach the given precision.

$$\begin{cases} 10x_1 + 2x_3 + 4x_4 = 110 \\ 2x_1 + 16x_2 - 3x_3 + 8x_4 = 128 \\ x_1 + 5x_2 + 11x_3 - 4x_4 = 102 \\ 8x_1 + x_2 + 6x_3 - 17x_4 = 81 \end{cases}$$

3.2 Method

The system is rewritten in the equivalent form $x = \beta + \alpha x$, where $\beta_i = b_i/a_{ii}$, $\alpha_{ij} = -a_{ij}/a_{ii}$ for $i \neq j$ and $\alpha_{ii} = 0$. Both methods start from $x^{(0)} = \beta$.

The simple iteration method computes $x^{(k+1)} = \beta + \alpha x^{(k)}$. The Seidel method uses the components that are already updated in the current iteration:

$$x_i^{(k+1)} = \beta_i + \sum_{j<i} \alpha_{ij} x_j^{(k+1)} + \sum_{j>i} \alpha_{ij} x_j^{(k)}.$$

If $\|\alpha\| < 1$, both methods converge for any initial vector, and the iterations stop when the a posteriori estimate of the error does not exceed ε:

$$\varepsilon^{(k)} = \frac{\|\alpha\|}{1 - \|\alpha\|} \|x^{(k)} - x^{(k-1)}\| \leq \varepsilon, \qquad \varepsilon^{(k)} = \frac{\|C\|}{1 - \|\alpha\|} \|x^{(k)} - x^{(k-1)}\| \leq \varepsilon,$$

for the simple iteration and the Seidel method respectively, where C is the upper triangular part of α ($c_{ij} = \alpha_{ij}$ for $j > i$). If $\|\alpha\| \geq 1$, the criterion is $\|x^{(k)} - x^{(k-1)}\| \leq \varepsilon$. All norms are $\|\cdot\|_\infty$. The a priori estimate of the number of simple iterations is

$$k + 1 \geq \frac{\lg \varepsilon - \lg \|\beta\| + \lg(1 - \|\alpha\|)}{\lg \|\alpha\|}.$$

A zero on the diagonal makes the transformation impossible. A non-finite iterate or 10^5 iterations without reaching the precision mean that the method does not converge. The program reports this for each method separately and fails only if neither method converges.

3.3 Results

The program reads n, the matrix A, the vector b and the precision ε, solves the system by both methods and then repeats the solution for $\varepsilon = 10^{-1}, \ldots, 10^{-10}$ to show how the number of iterations depends on the precision.

ε	A priori estimate	Simple iteration	Seidel
10^{-1}	74	12	7
10^{-2}	98	15	9
10^{-3}	122	18	10
10^{-4}	146	22	12
10^{-5}	171	26	14
10^{-6}	195	29	15
10^{-7}	219	33	17
10^{-8}	243	36	19
10^{-9}	267	40	20
10^{-10}	291	43	22

Table 1: Number of iterations depending on the precision

Listing 9: task3/input.txt

```
4
10 0 2 4
2 16 -3 8
1 5 11 -4
8 1 6 -17
110 128 102 81
0.0001
```

Listing 10: task3/output.txt

```
||alpha|| = 0.9091
A priori estimate: 146 iterations
Simple iteration: 22 iterations
x = (9.0000, 7.0000, 6.0000, 2.0000)
Seidel: 12 iterations
x = (9.0000, 7.0000, 6.0000, 2.0000)

Iterations depending on the precision:
    eps  a priori  simple  Seidel
  1e-01        74      12       7
  1e-02        98      15       9
  1e-03       122      18      10
  1e-04       146      22      12
  1e-05       171      26      14
  1e-06       195      29      15
  1e-07       219      33      17
```

```
1e-08       243       36       19
1e-09       267       40       20
1e-10       291       43       22
```

The exact solution is $x = (9, 7, 6, 2)^T$. The matrix is diagonally dominant and $\|\alpha\|_\infty = 10/11 \approx 0.9091$, so both methods converge.

3.4 Analysis

The number of iterations grows linearly in $\lg(1/\varepsilon)$: about 3.5 iterations per decimal digit for the simple iteration and about 1.7 for the Seidel method. The rate is determined by the spectral radius of the iteration matrix, $\rho(\alpha) \approx 0.518$, which gives $1/\lg(1/0.518) \approx 3.5$. For the Seidel method the spectral radius is $\rho \approx 0.257 \approx \rho(\alpha)^2$, so it needs about half as many iterations.

The a priori estimate is correct but about 7 times too pessimistic: it uses $\|\alpha\|_\infty \approx 0.909$, which is much larger than $\rho(\alpha)$. The estimate is useful to guarantee termination, while the a posteriori criterion stops the process in time.

The condition $\|\alpha\| < 1$ is only sufficient. For the matrix with ones on the diagonal and 0.5 elsewhere $\|\alpha\|_\infty = \rho(\alpha) = 1$: the error along $(1, 1, 1)^T$ changes its sign at every step and the simple iteration never converges, while the Seidel method converges, because the matrix is symmetric and positive definite. In this case the program reports `no convergence` for the simple iteration and prints the solution found by the Seidel method.

3.5 Source code

Listing 11: task3/iterative.py

```python
import math
from collections.abc import Callable
from dataclasses import dataclass

from lab1.common import (
    Matrix,
    Vector,
    distance_inf,
    is_square,
    matrix_norm_inf,
    matvec,
    norm_inf,
)

MAX_ITERATIONS = 100000
MAX_ESTIMATE = 2**64


@dataclass(frozen=True, slots=True)
class IterationForm:
```

```
    alpha: Matrix
    beta: Vector


@dataclass(frozen=True, slots=True)
class IterationResult:
    solution: Vector
    iterations: int
    converged: bool


def _upper_norm_inf(matrix: Matrix) -> float:
    result = 0.0
    for i, row in enumerate(matrix):
        row_sum = 0.0
        for value in row[i + 1 :]:
            row_sum += abs(value)
        result = max(result, row_sum)
    return result


def _iterate(
    form: IterationForm, eps: float, factor: float, step: Callable[[Vector], Vector]
) -> IterationResult:
    solution = form.beta
    iterations = 0
    while iterations < MAX_ITERATIONS:
        following = step(solution)
        iterations += 1
        if not all(math.isfinite(value) for value in following):
            return IterationResult(solution, iterations, converged=False)
        error = factor * distance_inf(following, solution)
        solution = following
        if error <= eps:
            return IterationResult(solution, iterations, converged=True)
    return IterationResult(solution, iterations, converged=False)


def to_iteration_form(matrix: Matrix, rhs: Vector) -> IterationForm:
    if not matrix or not is_square(matrix) or len(rhs) != len(matrix):
        raise ValueError("iterative methods need a square matrix and a matching rhs")
    n = len(matrix)
    alpha = [[0.0] * n for _ in range(n)]
    beta = [0.0] * n
    for i in range(n):
        diagonal = matrix[i][i]
        if diagonal == 0.0:
            raise ValueError("zero on the main diagonal")
        beta[i] = rhs[i] / diagonal
        for j in range(n):
            alpha[i][j] = 0.0 if i == j else -matrix[i][j] / diagonal
    return IterationForm(alpha, beta)


def simple_iteration(matrix: Matrix, rhs: Vector, eps: float) -> IterationResult:
    form = to_iteration_form(matrix, rhs)
    alpha_norm = matrix_norm_inf(form.alpha)
    factor = alpha_norm / (1.0 - alpha_norm) if alpha_norm < 1.0 else 1.0

    def step(current: Vector) -> Vector:
        following = matvec(form.alpha, current)
        for i in range(len(following)):
```

```
            following[i] += form.beta[i]
        return following

    return _iterate(form, eps, factor, step)


def seidel(matrix: Matrix, rhs: Vector, eps: float) -> IterationResult:
    form = to_iteration_form(matrix, rhs)
    alpha_norm = matrix_norm_inf(form.alpha)
    factor = _upper_norm_inf(form.alpha) / (1.0 - alpha_norm) if alpha_norm < 1.0 else 1.0

    def step(current: Vector) -> Vector:
        following = current[:]
        for i, row in enumerate(form.alpha):
            value = form.beta[i]
            for j, coefficient in enumerate(row):
                value += coefficient * following[j]
            following[i] = value
        return following

    return _iterate(form, eps, factor, step)


def apriori_iterations(matrix: Matrix, rhs: Vector, eps: float) -> int | None:
    form = to_iteration_form(matrix, rhs)
    alpha_norm = matrix_norm_inf(form.alpha)
    beta_norm = norm_inf(form.beta)
    if alpha_norm >= 1.0:
        return None
    if alpha_norm == 0.0 or beta_norm == 0.0:
        return 0
    bound = (math.log(eps) - math.log(beta_norm) + math.log(1.0 - alpha_norm)) / math.log(
        alpha_norm
    )
    if not math.isfinite(bound) or bound >= MAX_ESTIMATE:
        return None
    return 0 if bound <= 1.0 else math.ceil(bound - 1.0)
```

Listing 12: task3/main.py

```
import sys
from pathlib import Path

from lab1.common import (
    InputReader,
    Matrix,
    Vector,
    format_number,
    format_vector,
    matrix_norm_inf,
    run,
)
from lab1.task3.iterative import (
    IterationResult,
    apriori_iterations,
    seidel,
    simple_iteration,
    to_iteration_form,
)

INPUT = Path(__file__).with_name("input.txt")
```

```python
PRECISIONS = [10.0**-power for power in range(1, 11)]


def _result_lines(name: str, result: IterationResult) -> list[str]:
    if not result.converged:
        return [f"{name}: no convergence after {result.iterations} iterations"]
    return [f"{name}: {result.iterations} iterations", f"x = {format_vector(result.solution)}"]


def _count(result: IterationResult) -> str:
    return str(result.iterations) if result.converged else "-"


def _analysis(matrix: Matrix, rhs: Vector) -> list[str]:
    lines = ["Iterations depending on the precision:", " eps a priori simple Seidel"]
    for eps in PRECISIONS:
        estimate = apriori_iterations(matrix, rhs, eps)
        jacobi = _count(simple_iteration(matrix, rhs, eps))
        gauss_seidel = _count(seidel(matrix, rhs, eps))
        bound = "-" if estimate is None else str(estimate)
        lines.append(f"{eps:7.0e} {bound:>8} {jacobi:>6} {gauss_seidel:>6}")
    return lines


def task(reader: InputReader) -> list[str]:
    n = reader.size()
    matrix = reader.matrix(n)
    rhs = reader.vector(n)
    eps = reader.precision()
    reader.finish()

    form = to_iteration_form(matrix, rhs)
    jacobi = simple_iteration(matrix, rhs, eps)
    gauss_seidel = seidel(matrix, rhs, eps)
    estimate = apriori_iterations(matrix, rhs, eps)
    if not jacobi.converged and not gauss_seidel.converged:
        raise RuntimeError("neither method converged")

    return [
        f"||alpha|| = {format_number(matrix_norm_inf(form.alpha))}",
        f"A priori estimate: {'none' if estimate is None else f'{estimate} iterations'}",
        *_result_lines("Simple iteration", jacobi),
        *_result_lines("Seidel", gauss_seidel),
        "",
        *_analysis(matrix, rhs),
    ]


if __name__ == "__main__":
    sys.exit(run(task, INPUT, sys.argv[1:]))
```

4 Jacobi rotation method

4.1 Task

Implement the rotation method taking the matrix and the precision as input. Find the eigenvalues and the eigenvectors of the symmetric matrix and analyze the dependence of the error on the number of iterations.

$$A = \begin{pmatrix} 3 & 2 & 6 \\ 2 & -3 & -7 \\ 6 & -7 & 3 \end{pmatrix}$$

4.2 Method

The method builds a sequence of similar matrices $A^{(k+1)} = U^{(k)T} A^{(k)} U^{(k)}$, where $U^{(k)}$ is the rotation in the plane (i, j) of the largest off-diagonal element $|a_{ij}^{(k)}|$: $u_{ii} = u_{jj} = \cos\varphi_k$, $u_{ij} = -\sin\varphi_k$, $u_{ji} = \sin\varphi_k$. The angle makes the element $a_{ij}^{(k+1)}$ zero:

$$\varphi_k = \frac{1}{2}\arctan\frac{2a_{ij}^{(k)}}{a_{ii}^{(k)} - a_{jj}^{(k)}}, \qquad \varphi_k = \frac{\pi}{4}\operatorname{sign} a_{ij}^{(k)} \text{ if } a_{ii}^{(k)} = a_{jj}^{(k)}.$$

A rotation changes only rows and columns i and j, so one step costs $O(n)$ plus $O(n^2)$ for the search of the pivot. The error is measured by the norm of the off-diagonal part

$$t\big(A^{(k)}\big) = \Big(\sum_{i<j}\big(a_{ij}^{(k)}\big)^2\Big)^{1/2},$$

and the process stops when $t(A^{(k)}) \le \varepsilon$. The diagonal of $A^{(k)}$ contains the eigenvalues and the columns of $U = U^{(0)}U^{(1)}\cdots U^{(k-1)}$ are the orthonormal eigenvectors.

4.3 Results

The program reads n, the matrix A and the precision ε and prints the eigenvalues, the eigenvectors and the error after every rotation.

Listing 13: task4/input.txt

```
3
3 2 6
2 -3 -7
6 -7 3
0.000001
```

Listing 14: task4/output.txt

```
lambda_1 = 2.4775, x_1 = (0.7410, 0.6150, -0.2695)
lambda_2 = -9.7068, x_2 = (-0.3862, 0.7187, 0.5783)
lambda_3 = 10.2293, x_3 = (0.5494, -0.3244, 0.7700)

Rotations: 8
Error after every rotation:
  k  off(A)
  0  9.4340e+00
  1  6.3246e+00
  2  3.9081e+00
  3  1.4365e+00
  4  8.2361e-01
  5  4.8846e-02
  6  3.3034e-03
  7  2.0768e-05
  8  3.4412e-09
```

The unit tests check $Ax_i = \lambda_i x_i$ and $U^T U = E$ on random symmetric matrices up to 7×7.

4.4 Analysis

A rotation preserves the Frobenius norm and moves the weight of the element a_{ij} to the diagonal, so $t^2(A^{(k+1)}) = t^2(A^{(k)}) - \left(a_{ij}^{(k)}\right)^2$ and the error decreases monotonically (the first step of the table removes $a_{23} = -7$: $9.434^2 - 6.325^2 = 49$). Since the largest of the $n(n-1)/2$ elements is chosen, $a_{ij}^2 \geq \frac{2}{n(n-1)} t^2$, and the convergence is at least linear:

$$t\big(A^{(k)}\big) \leq \left(1 - \frac{2}{n(n-1)}\right)^{k/2} t\big(A^{(0)}\big).$$

For $n = 3$ the guaranteed factor is $\sqrt{2/3} \approx 0.82$ per rotation, but the table shows a faster decrease. The first four rotations reduce the error by 1.5–2.7 times, and then the

convergence becomes quadratic: $4.9 \cdot 10^{-2} \to 3.3 \cdot 10^{-3} \to 2.1 \cdot 10^{-5} \to 3.4 \cdot 10^{-9}$, the number of correct digits roughly doubles with every rotation. That is why 8 rotations are enough for $\varepsilon = 10^{-6}$, and a higher precision costs only one or two extra rotations.

4.5 Source code

Listing 15: task4/rotation.py

```python
import math
from dataclasses import dataclass

from lab1.common import Matrix, Vector, copy_matrix, identity, is_symmetric, matrix_norm_inf

MAX_ROTATIONS = 100000
SYMMETRY_TOLERANCE = 1e-12


@dataclass(frozen=True, slots=True)
class EigenSystem:
    values: Vector
    vectors: Matrix
    errors: Vector
    rotations: int


def _rotate_columns(matrix: Matrix, i: int, j: int, cosine: float, sine: float) -> None:
    for row in matrix:
        left = row[i]
        right = row[j]
        row[i] = cosine * left + sine * right
        row[j] = -sine * left + cosine * right


def _rotate_rows(matrix: Matrix, i: int, j: int, cosine: float, sine: float) -> None:
    top = matrix[i]
    bottom = matrix[j]
    for k in range(len(top)):
        upper = top[k]
        lower = bottom[k]
        top[k] = cosine * upper + sine * lower
        bottom[k] = -sine * upper + cosine * lower


def off_diagonal_norm(matrix: Matrix) -> float:
    total = 0.0
    for i, row in enumerate(matrix):
        for value in row[i + 1 :]:
            total += value * value
    return math.sqrt(total)


def rotation_method(matrix: Matrix, eps: float) -> EigenSystem:
    if not matrix or not is_symmetric(matrix, SYMMETRY_TOLERANCE * matrix_norm_inf(matrix)):
        raise ValueError("rotation method needs a symmetric matrix")
    n = len(matrix)
    a = copy_matrix(matrix)
    vectors = identity(n)
```

```
    errors = [off_diagonal_norm(a)]
    rotations = 0

    while errors[-1] > eps:
        if rotations == MAX_ROTATIONS:
            raise RuntimeError("rotation method did not converge")
        pivot_row = 0
        pivot_col = 1
        for i in range(n):
            for j in range(i + 1, n):
                if abs(a[i][j]) > abs(a[pivot_row][pivot_col]):
                    pivot_row = i
                    pivot_col = j
        pivot = a[pivot_row][pivot_col]
        difference = a[pivot_row][pivot_row] - a[pivot_col][pivot_col]
        angle = (
            math.copysign(math.pi / 4.0, pivot)
            if difference == 0.0
            else 0.5 * math.atan(2.0 * pivot / difference)
        )
        cosine = math.cos(angle)
        sine = math.sin(angle)
        _rotate_columns(a, pivot_row, pivot_col, cosine, sine)
        _rotate_rows(a, pivot_row, pivot_col, cosine, sine)
        a[pivot_row][pivot_col] = 0.0
        a[pivot_col][pivot_row] = 0.0
        _rotate_columns(vectors, pivot_row, pivot_col, cosine, sine)
        rotations += 1
        errors.append(off_diagonal_norm(a))

    return EigenSystem([a[i][i] for i in range(n)], vectors, errors, rotations)
```

Listing 16: task4/main.py

```
import sys
from pathlib import Path

from lab1.common import InputReader, column, format_number, format_scientific, format_vector, run
from lab1.task4.rotation import rotation_method

INPUT = Path(__file__).with_name("input.txt")


def task(reader: InputReader) -> list[str]:
    n = reader.size()
    matrix = reader.matrix(n)
    eps = reader.precision()
    reader.finish()

    eigen = rotation_method(matrix, eps)

    return [
        *(
            f"lambda_{i + 1} = {format_number(value)}, "
            f"x_{i + 1} = {format_vector(column(eigen.vectors, i))}"
            for i, value in enumerate(eigen.values)
        ),
        "",
        f"Rotations: {eigen.rotations}",
        "Error after every rotation:",
        " k off(A)",
```

```
        *(f"{k:>3} {format_scientific(error)}" for k, error in enumerate(eigen.errors)),
    ]


if __name__ == "__main__":
    sys.exit(run(task, INPUT, sys.argv[1:]))
```

5 QR algorithm

5.1 Task

Implement the QR decomposition of a matrix. Based on it, implement the QR algorithm for the full eigenvalue problem of an arbitrary matrix taking the matrix and the precision as input, and find the eigenvalues of the matrix.

$$A = \begin{pmatrix} -6 & -3 & 7 \\ 9 & 1 & -6 \\ 3 & -5 & 5 \end{pmatrix}$$

5.2 Method

The QR decomposition $A = QR$ with an orthogonal Q and an upper triangular R is built with Householder reflections. At step $k = 1, \ldots, n-1$ the vector v is formed from the part of column k on and below the diagonal:

$$v = a_k + \operatorname{sign}(a_{kk})\|a_k\|_2 e_k, \qquad H_k = E - \frac{2vv^T}{v^T v}.$$

The reflection H_k zeroes the elements below the diagonal in column k, and the choice of the sign avoids cancellation. Then $R = H_{n-1} \cdots H_1 A$ and $Q = H_1 \cdots H_{n-1}$. The reflections are applied as rank-one updates without forming H_k.

The QR algorithm builds the sequence $A^{(0)} = A$, $A^{(k)} = Q^{(k)}R^{(k)}$, $A^{(k+1)} = R^{(k)}Q^{(k)} = Q^{(k)T}A^{(k)}Q^{(k)}$ of similar matrices. It converges to a block upper triangular form: 1×1 blocks hold real eigenvalues, 2×2 blocks hold pairs of complex conjugate eigenvalues or eigenvalues of equal or close moduli. The diagonal is scanned column by column:

- if $\left(\sum_{i>j} a_{ij}^2\right)^{1/2} \le \varepsilon$, then a_{jj} is a real eigenvalue;
- otherwise columns $j, j+1$ form a block, whose eigenvalues are the roots of $(a_{jj} - \lambda)(a_{j+1,j+1} - \lambda) = a_{j,j+1}a_{j+1,j}$. The block is accepted when the elements below it are smaller than ε and its eigenvalues differ from those of the previous iteration by at most ε.

The iterations stop when every diagonal position is covered by an accepted block. Once the elements below a block vanish, the eigenvalues of the block are exact eigenvalues of A, even if the block itself has not split yet.

5.3 Results

The program reads n, the matrix A and the precision ε and prints the QR decomposition of A, the eigenvalues, the number of iterations and the number of iterations for $\varepsilon = 10^{-1}, \ldots, 10^{-10}$.

Listing 17: task5/input.txt

```
3
-6 -3 7
9 1 -6
3 -5 5
0.000001
```

Listing 18: task5/output.txt

```
Q =
  -0.5345  -0.4174  -0.7349
   0.8018   0.0246  -0.5971
   0.2673  -0.9084   0.3215
R =
  11.2250   1.0690  -7.2161
   0.0000   5.8187  -7.6109
   0.0000   0.0000   0.0459

lambda_1 = 7.4433
lambda_2 = -7.3888
lambda_3 = -0.0545
Iterations: 5

Iterations depending on the precision:
    eps  iterations
  1e-01           2
  1e-02           3
  1e-03           3
  1e-04           4
  1e-05           4
  1e-06           5
  1e-07           5
  1e-08           6
  1e-09           6
  1e-10           7
```

All three eigenvalues are real: $\lambda_1 \approx 7.4433$, $\lambda_2 \approx -7.3888$, $\lambda_3 \approx -0.0545$. Their sum is $0 = \operatorname{tr} A$ and their product is $3 = \det A$. The unit tests check $QR = A$ and $Q^T Q = E$, find the purely imaginary pair of a rotation matrix, compare the eigenvalues of symmetric matrices with the rotation method and check the trace and the determinant of random matrices up to 6×6.

5.4 Analysis

The rate at which the element $a_{i+1,i}$ vanishes is $|\lambda_{i+1}/\lambda_i|^k$. In the variant these ratios are very different: $|\lambda_3/\lambda_2| \approx 0.0074$ and $|\lambda_2/\lambda_1| \approx 0.9927$.

k	$\lvert a_{21}^{(k)} \rvert$	$\lvert a_{32}^{(k)} \rvert$	$a_{11}^{(k)}$
1	2.631	$4.2 \cdot 10^{-2}$	-7.0714
2	2.522	$3.2 \cdot 10^{-4}$	-6.8193
3	2.624	$2.3 \cdot 10^{-6}$	-7.0404
5	2.661	$1.2 \cdot 10^{-10}$	-7.0284
1000	$5.4 \cdot 10^{-2}$	0	7.4406
2000	$3.4 \cdot 10^{-5}$	0	7.4433
3000	$2.2 \cdot 10^{-8}$	0	7.4433

Table 2: Sub-diagonal elements of $A^{(k)}$

The element a_{32} decreases about 130 times per iteration, so λ_3 is separated after a few iterations. The element a_{21} stays near 2.5 for hundreds of iterations and then decreases only by a factor of 10^3 per 1000 iterations, and the diagonal elements of the block keep changing. If the algorithm waited for a_{21} itself, $\varepsilon = 10^{-6}$ would require 2482 iterations. The block criterion stops as soon as λ_3 is isolated and the eigenvalues of the 2×2 block stop changing:

ε	10^{-1}	10^{-2}	10^{-3}	10^{-4}	10^{-6}	10^{-8}	10^{-10}
Iterations	2	3	3	4	5	6	7

Table 3: Number of QR iterations depending on the precision

Every iteration multiplies a_{32} by about 0.0074, so one more iteration gives about two more correct digits, in agreement with the table.

5.5 Source code

Listing 19: task5/qr.py

```
1 || import cmath
```

```python
import math
from dataclasses import dataclass

from lab1.common import Matrix, copy_matrix, identity, is_square, matmul

MAX_ITERATIONS = 100000

ComplexPair = tuple[complex, complex]


@dataclass(frozen=True, slots=True)
class QrDecomposition:
    q: Matrix
    r: Matrix


@dataclass(frozen=True, slots=True)
class QrEigenvalues:
    values: list[complex]
    iterations: int


def _column_tail(matrix: Matrix, col: int, from_row: int) -> float:
    total = 0.0
    for row in matrix[from_row:]:
        total += row[col] * row[col]
    return math.sqrt(total)


def _block_eigenvalues(matrix: Matrix, j: int) -> ComplexPair:
    a = matrix[j][j]
    b = matrix[j][j + 1]
    c = matrix[j + 1][j]
    d = matrix[j + 1][j + 1]
    half_trace = (a + d) / 2.0
    root = cmath.sqrt(complex((a - d) * (a - d) / 4.0 + b * c, 0.0))
    return half_trace + root, half_trace - root


def _is_stable(current: ComplexPair, last: ComplexPair | None, eps: float) -> bool:
    if last is None:
        return False
    return abs(current[0] - last[0]) <= eps and abs(current[1] - last[1]) <= eps


def _extract_eigenvalues(
    matrix: Matrix, eps: float, previous: list[ComplexPair | None]
) -> list[complex] | None:
    n = len(matrix)
    values: list[complex] = []
    converged = True
    j = 0
    while j < n:
        if j + 1 == n or _column_tail(matrix, j, j + 1) <= eps:
            values.append(complex(matrix[j][j], 0.0))
            j += 1
            continue
        current = _block_eigenvalues(matrix, j)
        isolated = (
            _column_tail(matrix, j, j + 2) <= eps and _column_tail(matrix, j + 1, j + 2) <= eps
        )
        stable = _is_stable(current, previous[j], eps)
```

```
        previous[j] = current
        converged = converged and isolated and stable
        values.extend(current)
        j += 2
    return values if converged else None


def decompose_qr(matrix: Matrix) -> QrDecomposition:
    if not matrix or not is_square(matrix):
        raise ValueError("QR decomposition needs a non-empty square matrix")
    n = len(matrix)
    q = identity(n)
    r = copy_matrix(matrix)
    for k in range(n - 1):
        norm = _column_tail(r, k, k)
        if norm == 0.0:
            continue
        v = [0.0] * n
        for i in range(k, n):
            v[i] = r[i][k]
        v[k] += math.copysign(norm, v[k])
        squared_norm = 0.0
        for i in range(k, n):
            squared_norm += v[i] * v[i]
        for j in range(n):
            dot = 0.0
            for i in range(k, n):
                dot += v[i] * r[i][j]
            factor = 2.0 * dot / squared_norm
            for i in range(k, n):
                r[i][j] -= factor * v[i]
        for row in q:
            dot = 0.0
            for j in range(k, n):
                dot += row[j] * v[j]
            factor = 2.0 * dot / squared_norm
            for j in range(k, n):
                row[j] -= factor * v[j]
        for i in range(k + 1, n):
            r[i][k] = 0.0
    return QrDecomposition(q, r)


def qr_algorithm(matrix: Matrix, eps: float) -> QrEigenvalues:
    if not matrix or not is_square(matrix):
        raise ValueError("QR algorithm needs a non-empty square matrix")
    a = copy_matrix(matrix)
    previous: list[ComplexPair | None] = [None] * len(matrix)
    for iteration in range(MAX_ITERATIONS + 1):
        values = _extract_eigenvalues(a, eps, previous)
        if values is not None:
            return QrEigenvalues(values, iteration)
        qr = decompose_qr(a)
        a = matmul(qr.r, qr.q)
    raise RuntimeError("QR algorithm did not converge")
```

Listing 20: task5/main.py

```
import sys
from pathlib import Path

```

```python
from lab1.common import InputReader, format_complex, format_matrix, run
from lab1.task5.qr import decompose_qr, qr_algorithm

INPUT = Path(__file__).with_name("input.txt")
PRECISIONS = [10.0**-power for power in range(1, 11)]


def task(reader: InputReader) -> list[str]:
    n = reader.size()
    matrix = reader.matrix(n)
    eps = reader.precision()
    reader.finish()

    qr = decompose_qr(matrix)
    eigen = qr_algorithm(matrix, eps)

    return [
        "Q =",
        *format_matrix(qr.q),
        "R =",
        *format_matrix(qr.r),
        "",
        *(f"lambda_{i + 1} = {format_complex(value)}" for i, value in enumerate(eigen.values)),
        f"Iterations: {eigen.iterations}",
        "",
        "Iterations depending on the precision:",
        " eps iterations",
        *(f"{value:7.0e} {qr_algorithm(matrix, value).iterations:>10}" for value in PRECISIONS),
    ]


if __name__ == "__main__":
    sys.exit(run(task, INPUT, sys.argv[1:]))
```

6 Program structure

The programs are written in Python 3.13 with the standard library only: every method is implemented from scratch, without NumPy. Every task has its own folder `task1`, ..., `task5` with the same files:

- the module of the method (`lu.py`, `tridiagonal.py`, `iterative.py`, `rotation.py`, `qr.py`);
- `main.py`, which reads `input.txt` next to it (or the file given as an argument), solves the task and prints the result: `python3 -m lab1.task3.main`;
- `input.txt` with the data of the variant and `output.txt` with the expected output, which the tests compare with the actual one.

The file `lab1/main.py` runs all five tasks one after another. All tests are in the folder `lab1/tests`: one file per method, the tests of `common.py` and the tests that run every program and compare its output with `output.txt`. The shared module `common.py` contains the matrix operations (a matrix is a list of rows, a vector is a list of numbers), the class `InputReader` that reads the size, the numbers and the precision from the input file, the output formatting with four digits after the decimal point and the function `run`. It computes the whole result before printing it, so an error never leaves a partial output; errors are printed as `error: <message>` with the exit code 1. All sums are accumulated in explicit loops: the built-in `sum` uses compensated summation since Python 3.12 and would change the last bits of the results.

Listing 21: main.py

```
import sys

from lab1.common import run
from lab1.task1 import main as task1
from lab1.task2 import main as task2
from lab1.task3 import main as task3
from lab1.task4 import main as task4
from lab1.task5 import main as task5

TASKS = [
    ("1.1", "LU decomposition", task1),
    ("1.2", "Thomas algorithm", task2),
    ("1.3", "Simple iteration and Seidel methods", task3),
    ("1.4", "Jacobi rotation method", task4),
    ("1.5", "QR algorithm", task5),
]


def main() -> int:
    status = 0
    for index, (number, title, module) in enumerate(TASKS):
        if index > 0:
```

```
            print()
        print(f"Task {number}. {title}")
        status = max(status, run(module.task, module.INPUT, []))
    return status


if __name__ == "__main__":
    sys.exit(main())
```

Listing 22: common.py

```
import math
import sys
from collections.abc import Callable
from pathlib import Path

Vector = list[float]
Matrix = list[list[float]]

PRECISION = 4
ZERO_THRESHOLD = 5e-5


def zeros(rows: int, cols: int) -> Matrix:
    return [[0.0] * cols for _ in range(rows)]


def identity(size: int) -> Matrix:
    result = zeros(size, size)
    for i in range(size):
        result[i][i] = 1.0
    return result


def copy_matrix(matrix: Matrix) -> Matrix:
    return [row[:] for row in matrix]


def is_square(matrix: Matrix) -> bool:
    return all(len(row) == len(matrix) for row in matrix)


def column(matrix: Matrix, col: int) -> Vector:
    return [row[col] for row in matrix]


def transposed(matrix: Matrix) -> Matrix:
    cols = len(matrix[0]) if matrix else 0
    return [[row[j] for row in matrix] for j in range(cols)]


def matmul(lhs: Matrix, rhs: Matrix) -> Matrix:
    if any(len(row) != len(rhs) for row in lhs):
        raise ValueError("matrix sizes do not match")
    cols = len(rhs[0]) if rhs else 0
    result = zeros(len(lhs), cols)
    for row, out in zip(lhs, result, strict=True):
        for value, other in zip(row, rhs, strict=True):
            for j in range(cols):
                out[j] += value * other[j]
    return result
```

```
def matvec(matrix: Matrix, vector: Vector) -> Vector:
    if any(len(row) != len(vector) for row in matrix):
        raise ValueError("matrix and vector sizes do not match")
    result = [0.0] * len(matrix)
    for i, row in enumerate(matrix):
        for value, component in zip(row, vector, strict=True):
            result[i] += value * component
    return result


def norm_inf(vector: Vector) -> float:
    result = 0.0
    for value in vector:
        result = max(result, abs(value))
    return result


def matrix_norm_inf(matrix: Matrix) -> float:
    result = 0.0
    for row in matrix:
        row_sum = 0.0
        for value in row:
            row_sum += abs(value)
        result = max(result, row_sum)
    return result


def distance_inf(lhs: Vector, rhs: Vector) -> float:
    if len(lhs) != len(rhs):
        raise ValueError("vector sizes do not match")
    result = 0.0
    for left, right in zip(lhs, rhs, strict=True):
        result = max(result, abs(left - right))
    return result


def is_symmetric(matrix: Matrix, tolerance: float = 0.0) -> bool:
    if not is_square(matrix):
        return False
    n = len(matrix)
    return all(
        abs(matrix[i][j] - matrix[j][i]) <= tolerance for i in range(n) for j in range(i + 1, n)
    )


class InputReader:
    def __init__(self, text: str) -> None:
        self._tokens = text.split()
        self._position = 0

    def _next(self) -> str:
        if self._position == len(self._tokens):
            raise ValueError("unexpected end of input")
        token = self._tokens[self._position]
        self._position += 1
        return token

    def size(self) -> int:
        token = self._next()
        if not token.isdigit() or not 1 <= int(token) <= 1000:
```

```
            raise ValueError(f"expected a matrix size from 1 to 1000, got '{token}'")
        return int(token)

    def number(self) -> float:
        token = self._next()
        try:
            value = float(token)
        except ValueError:
            raise ValueError(f"expected a number, got '{token}'") from None
        if not math.isfinite(value) or abs(value) > 1e100:
            raise ValueError(f"numbers must lie in [-1e100, 1e100], got '{token}'")
        return value

    def precision(self) -> float:
        value = self.number()
        if value <= 0.0:
            raise ValueError("precision must be positive")
        return value

    def vector(self, size: int) -> Vector:
        return [self.number() for _ in range(size)]

    def matrix(self, size: int) -> Matrix:
        return [self.vector(size) for _ in range(size)]

    def finish(self) -> None:
        if self._position < len(self._tokens):
            raise ValueError(f"unexpected trailing input '{self._tokens[self._position]}'")


def format_number(value: float) -> str:
    return f"{0.0 if abs(value) < ZERO_THRESHOLD else value:.{PRECISION}f}"


def format_scientific(value: float) -> str:
    return f"{value:.{PRECISION}e}"


def format_complex(value: complex) -> str:
    if abs(value.imag) < ZERO_THRESHOLD:
        return format_number(value.real)
    sign = " - " if value.imag < 0.0 else " + "
    return f"{format_number(value.real)}{sign}{format_number(abs(value.imag))}i"


def format_vector(vector: Vector) -> str:
    return "(" + ", ".join(format_number(value) for value in vector) + ")"


def format_matrix(matrix: Matrix) -> list[str]:
    cells = [[format_number(value) for value in row] for row in matrix]
    width = max((len(cell) for row in cells for cell in row), default=0)
    return ["".join(f" {cell:>{width}}" for cell in row) for row in cells]


Task = Callable[[InputReader], list[str]]


def solve_file(task: Task, path: Path) -> list[str]:
    return task(InputReader(path.read_text(encoding="utf-8")))


```

```
def run(task: Task, default_input: Path, args: list[str]) -> int:
    path = Path(args[0]) if args else default_input
    try:
        lines = solve_file(task, path)
    except (OSError, ValueError, ArithmeticError, RuntimeError) as error:
        print(f"error: {error}", file=sys.stderr)
        return 1
    print("\n".join(lines))
    return 0
```

7 Conclusions

In this laboratory work for the course “Numerical Methods” I implemented the basic methods of numerical linear algebra and applied them to variant 28.

Direct methods give the solution in a fixed number of operations. The LU decomposition with partial pivoting solves a general system in $O(n^3)$, and the same factorization gives the determinant and the inverse matrix. The Thomas algorithm uses the structure of a tridiagonal matrix and solves the system in $O(n)$; diagonal dominance with non-zero off-diagonal elements guarantees its stability.

Iterative methods trade exactness for simplicity of every step. For the diagonally dominant system of the variant the Seidel method needed about half as many iterations as the simple iteration method, in agreement with the ratio of the spectral radii of their iteration matrices, while the a priori estimate turned out to be about 7 times too pessimistic.

The rotation method finds the complete eigensystem of a symmetric matrix: the error decreases monotonically and at the end converges quadratically. The QR algorithm with Householder reflections works for arbitrary matrices; its rate depends on the ratios of the moduli of the eigenvalues. In the variant two eigenvalues have almost equal moduli, and a full triangularization would take 2482 iterations, while treating them as a 2×2 block (the same way as a complex pair) gives the answer in 5 iterations.

All results were verified by unit tests on random matrices (residuals, orthogonality, trace and determinant) and by end-to-end tests of the programs.

References

[1] Golub G. H., Van Loan C. F. *Matrix Computations, 4th edition.* — Johns Hopkins University Press, 2013.

[2] Demmel J. W. *Applied Numerical Linear Algebra.* — SIAM, 1997.

[3] Burden R. L., Faires J. D. *Numerical Analysis, 9th edition.* — Brooks/Cole, 2011.